UNIVERSIDAD DE PINAR DEL RÍO "HERMANOS SAÍZ MONTES DE OCA" • FACULTAD DE CIENCIAS TÉCNICAS

Control de Biblioteca Central 'Bibliotrack'

Examen Parcial Individual — Implementación Propia de Lista Dinámica (ArrayList)

ESTUDIANTE EVALUADO (NRO. 8)
Eduardo Alejandro Silva Hernández

CARRERA Y AÑO ACADÉMICO
Ingeniería Informática — 2do Año (CPE)

ASIGNATURA
Estructuras de Datos I (Curso 2026–2027)

MÉTODO ASIGNADO
`public int eliminarLibrosSinEjemplares()`

1. ¿Qué problema resuelve nuestro proyecto en 'Bibliotrack'?

Objetivo del Ejercicio #8

En la biblioteca central necesitamos limpiar el catálogo quitando los libros que ya no tienen copias disponibles para préstamo:

- Guardar los libros en nuestra propia clase **ArrayList<Libro>** vista en conferencia.
- Recorrer la lista e identificar todo libro donde **ejemplaresDisponibles <= 0**.
- Remover esos registros del catálogo y devolver un entero (**int**) con el total de libros eliminados.

Estructura POO del proyecto: Dividido en 5 archivos: `List.java`, `ArrayList.java`, `Libro.java`, `Biblioteca.java` y `Main.java`.

Entidad de Datos: Clase Libro

Representa cada libro de la biblioteca con los 5 atributos asignados en la guía:

```java
public class Libro {
    private String isbn;
    private String titulo;
    private String autor;
    private int anioPublicacion;
    private int ejemplaresDisponibles;

    // Constructor y métodos Getters
    public int getEjemplaresDisponibles() {
        return ejemplaresDisponibles;
    }
}
```

2. ¿Cómo funciona por dentro nuestro propio ArrayList<E>?

Base Interna (Sin usar java.util)

- Implementa la interfaz **List<E>** con los métodos: `add`, `remove`, `get`, `size`, `clear` e `isEmpty`.
- **Object[] items:** Arreglo interno donde se guardan los elementos.
- **capacity = 10:** Capacidad inicial del arreglo al crear la lista.
- **size = 0:** Cantidad real de libros guardados actualmente.

Crecimiento Dinámico: isFull() y Expand()

Al agregar con `add(E e)`, verifica si `capacity == size`. Si está lleno, `Expand()` crea un arreglo con **+10 posiciones** y copia los datos con `System.arraycopy`.

¿Qué hace remove(int index) al borrar?

```java
public E remove(int index) {
    if (index < size) {
        E aux = (E) items[index];
        for (int i = index; i < size - 1; i++) {
            items[i] = items[i + 1];
        }
        size--;
        return aux;
    }
    return null;
}
```

Efecto clave en memoria: Al borrar la posición `index`, el ciclo `for` desplaza todos los elementos siguientes **un lugar hacia la izquierda** (`items[i] = items[i + 1]`) y resta 1 a `size`.

3. Implementación de eliminarLibrosSinEjemplares()

```java
public int eliminarLibrosSinEjemplares() {
    int contadorEliminados = 0;

    if (this.catalogo.isEmpty()) {
        return 0;
    }

    int i = 0;
    while (i < this.catalogo.size()) {
        Libro libroActual = this.catalogo.get(i);

        if (libroActual.getEjemplaresDisponibles() <= 0) {
            this.catalogo.remove(i);
            contadorEliminados++;
            // No hacemos i++ porque el siguiente
            // libro se desplazó a esta posición 'i'
        } else {
            i++; // Solo avanzamos si NO se borró
        }
    }
    return contadorEliminados;
}
```

Explicación Paso a Paso

- **Validación inicial:** Si la lista está vacía (`isEmpty()`), retorna `0` directamente.
- **Recorrido con get(i):** Inspeccionamos cada libro mientras `i < catalogo.size()`.
- **Condición <= 0:** Filtra libros con `0` copias o valores negativos.

¿Por qué NO incrementamos `i++` al eliminar?

Porque al ejecutar `remove(i)`, el libro que estaba en `i + 1` pasa a ocupar la casilla `i`. Si hiciéramos `i++`, nos saltaríamos la revisión de ese libro cuando hay dos agotados seguidos.

DIAPOSITIVA 5 DE 5 • VALIDACIÓN EN MAIN.JAVA

4. Casos de Prueba Normales y Casos Límite

Escenario en Main	Datos de Entrada en el ArrayList	Comportamiento	Resultado
1. Lista vacía (Caso límite inicial)	Catálogo recién creado sin libros (`size = 0`).	Detecta `isEmpty() == true` y retorna sin errores.	**Eliminados: 0**
2. Todos disponibles (Sin eliminaciones)	2 libros con ejemplares positivos (5 y 3 copias).	Recorre ambos con `i++` sin borrar ninguno.	**Eliminados: 0** Quedan: 2
3. Caso completo y bordes (Inicio, seguidos y final)	6 libros con ejemplares: `[0, 4, 0, -1, 6, 0]`. Incluye agotado al inicio, dos seguidos en medio y al final.	Borra los 4 agotados sin saltarse el `-1` consecutivo.	**Eliminados: 4** Quedan: 2
4. Vaciado total (Todos agotados)	2 libros agotados consecutivos con `0` ejemplares.	Remueve todos y comprueba `isEmpty() == true`.	**Eliminados: 2** Quedan: 0

Conclusión de las pruebas: El algoritmo maneja correctamente listas vacías, elementos en los bordes (primero y último), valores negativos y eliminaciones consecutivas usando únicamente el `ArrayList` de la asignatura.